

Minicurso - Audio e Transformers

```
1 # coding: utf-8
2 from odoo.exceptions import ValidationError
3 from datetime import datetime
4 import calendar
5 import math
6 import pytz
7 import io, base64
8
9 class AdmissionExtensionOnlineController(http.Controller):
10
11     @http.route('/get/type_wise_program', website=True, auth='none')
12     def type_wise_program(self, **kwargs):
13         if len(kwargs['types']) <= 0:
14             return "None"
15
16         types = kwargs['types']
17         program_list = []
18         domain = []
19
20         if types == 'local_bachelor_program_hsc':
21             domain = [('course_id.is_local_bachelor_program_hsc', '=', 1)]
22         elif types == 'local_bachelor_program_a_level':
23             domain = [('course_id.is_local_bachelor_program_a_level', '=', 1)]
24         elif types == 'local_bachelor_program_diploma':
25             domain = [('course_id.is_local_bachelor_program_diploma', '=', 1)]
26         elif types == 'local_masters_program_bachelor':
27             domain = [('course_id.is_local_masters_program_bachelor', '=', 1)]
28         elif types == 'international_bachelor_program':
29             domain = [('course_id.is_international_bachelor_program', '=', 1)]
30         elif types == 'international_masters_program':
31             domain = [('course_id.is_international_masters_program', '=', 1)]
32
33         domain.append(('state', '=', 'application'))
34         admission_register_list = http.request._request.env['admission.register'].search(domain)
35         for program in admission_register_list:
```

O som como sinal

- ▶ Variação de pressão em meio elástico → sinal digital.
- ▶ Domínios: tempo (forma de onda) e frequência (espectro).
- ▶ Conceitos: frequência (Hz), amplitude, espectro.
- ▶ Nyquist: $f_s \geq 2f_{\max}$ (evita aliasing).

Do clássico ao profundo

- ▶ **Handcrafted features:** MFCC, ZCR, Energia, Entropia.
- ▶ **Limitações:** pouca robustez a ruído, pouco contexto longo.
- ▶ **Representation learning:** CNN/RNN → Transformers.

Por que Transformers em áudio?

- ▶ Dependências de *longo alcance* via atenção.
- ▶ Paralelizáveis e naturalmente multimodais.
- ▶ Pré-treinados *self-supervised* + *transfer learning*.

Arquitetura geral

- ▶ Encoder / Decoder; Multi-Head Attention; FFN + residual + LayerNorm.
- ▶ Embeddings posicionais: ordem temporal sem recorrência.

Figura sugerida: diagrama de um Encoder Transformer com blocos MHA/FFN.

Atenção: fórmula essencial

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^{\top}}{\sqrt{d_k}}\right) V$$

- ▶ Q : queries, K : keys, V : values.
- ▶ Escala $1/\sqrt{d_k}$ para estabilidade numérica.
- ▶ Similaridade $\sim$ cosseno $\rightarrow$ pesos de foco contextual.

Pré-treinamento e adaptação

- ▶ **Self-supervised:** MLM/CTC, reconstrução mascarada.
- ▶ **Fine-tuning:** congelamento parcial, *linear probing*.
- ▶ **LoRA/Adapters:** adaptação leve e eficiente.

Padronização

- ▶ Mono, 16 kHz, PCM 16-bit.
- ▶ Normalização de amplitude ($[-1, 1]$).
- ▶ Segmentação (2–4 s) + *padding* para tamanhos fixos.

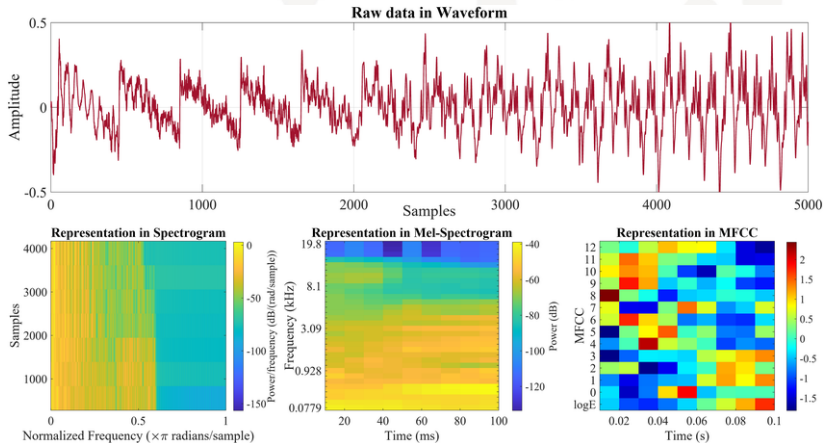
Visualização com librosa

```
import librosa, librosa.display, matplotlib.pyplot as plt

y, sr = librosa.load("sample.wav", sr=16000)
plt.figure(figsize=(10, 3))
librosa.display.waveshow(y, sr=sr)
plt.title("Forma de onda - sample.wav")
plt.show()
```

Mel-espectrograma

```
S = librosa.feature.melspectrogram(y=y, sr=sr, n_mels=128)
librosa.display.specshow(librosa.power_to_db(S, ref=np.max),
                          y_axis='mel', x_axis='time', sr=sr)
```



Linha do tempo — principais modelos

Modelo	Ano	Base	Tipo	Tarefa
Wav2Vec	2019	CNN+Transf	Self-sup.	Fala
HuBERT	2021	Transf	Self-sup.	Fala
WavLM	2022	Transf	Multi-task	Fala
Whisper	2022	Enc-Dec	ASR multilingue	ASR
AudioMAE	2023	ViT	Masked Modeling	Áudio geral
CLAP	2023	Dual Enc	Contraste Áudio-Texto	Multimodal

Extraindo embeddings (Wav2Vec2)

```
from transformers import Wav2Vec2Processor, Wav2Vec2Model
import torch, librosa, numpy as np

processor = Wav2Vec2Processor.from_pretrained("facebook/wav2vec2-base")
model = Wav2Vec2Model.from_pretrained("facebook/wav2vec2-base")

y, sr = librosa.load("voz_exemplo.wav", sr=16000)
inputs = processor(y, sampling_rate=sr, return_tensors="pt", padding=True)
with torch.no_grad():
    outputs = model(**inputs)
embeddings = outputs.last_hidden_state.mean(dim=1).numpy()
```

t-SNE / UMAP

```
from sklearn.manifold import TSNE
X2 = TSNE(n_components=2, random_state=42).fit_transform(embeddings)
plt.scatter(X2[:,0], X2[:,1]); plt.title("t-SNE de Embeddings")
```

Interpretação: clusters por timbre/fonética/emoção;
separabilidade real/fake.

Métricas de clusterização

```
from sklearn.metrics import silhouette_score, davies_bouldin_score
print("Silhouette:", silhouette_score(X2, labels))
print("Davies-Bouldin:", davies_bouldin_score(X2, labels))
```

ASR com Whisper

```
from transformers import pipeline  
asr = pipeline("automatic-speech-recognition", model="openai/whisper-small")  
asr("fala_exemplo.wav")
```

Patologias vocais

- ▶ Fine-tuning leve sobre embeddings fixos.
- ▶ Classificação: saudável vs patológico (AUC, F1, EER, Brier).
- ▶ Complemento com jitter/shimmer/HNR.

Deepfakes em voz

- ▶ Fusão: CQCC/MFCC + WavLM/HuBERT; XGBoost/SVM/RF.
- ▶ Calibração: Platt / Isotônica; EER, AUC, ECE/Brier.
- ▶ XAI: SHAP, atenção, Grad-CAM em espectrogramas.

CLAP — áudio & texto

```
from transformers import ClapProcessor, ClapModel
# model = ClapModel.from_pretrained("laion/clap-htsat-unfused")
# processor = ClapProcessor.from_pretrained("laion/clap-htsat-unfused")
```

Uso: busca semântica de sons, legendagem de áudio, organização por texto.

Mapas de atenção

- ▶ ' $\text{output_attentions} = \text{True}$ ' $\rightarrow$ matrizes (camada, cabeça, T, T).
- ▶ Padrões locais (camadas iniciais) vs contexto longo (finais).
- ▶ *Cuidado*: atenção alta $\neq$ causalidade.

SHAP / Grad-CAM

- ▶ SHAP: impacto das features em classificadores sobre embeddings.
- ▶ Grad-CAM: regiões tempo–frequência com CNNs de espectrograma.
- ▶ Curvas de confiança ao longo do tempo (chunking).

Panorama de métricas

Tipo	Métrica	Interpretação
Discriminação	AUC, EER	Separabilidade
Calibração	Brier, ECE	Probabilidades coerentes
Agrupamento	Silhouette, DB	Qualidade de clusters
ASR	WER, CER	Erro de transcrição

Fundação multimodal

- ▶ AudioGPT, SpeechLM, Gemini — unificação de fala, texto e visão.
- ▶ Aprendizado contrastivo (CLAP/AudioCLIP) e geração (MusicLM/AudioLDM).

Adaptação eficiente e *continual learning*

- ▶ LoRA, Adapters, Prefix/Prompt-tuning.
- ▶ DAFT, EWC, Replay — evitar *forgetting* entre domínios.

Impacto social

- ▶ Segurança: anti-spoofing e verificação de locutor.
- ▶ Saúde: triagem vocal remota e apoio clínico.
- ▶ Acessibilidade: TTS personalizado, legendas e tradução em tempo real.

Referências selecionadas

- ▶ Baevski et al. (2020). *wav2vec 2.0*. NeurIPS.
- ▶ Hsu et al. (2021). *HuBERT*. IEEE TASLP.
- ▶ Chen et al. (2022). *WavLM*. IEEE J-STSP.
- ▶ Radford et al. (2023). *Whisper*. arXiv:2212.04356.
- ▶ Wu et al. (2023). *AudioMAE*. arXiv:2207.06405.
- ▶ Elizalde et al. (2023). *CLAP*. ICASSP.